ROADMAP + PROJECT, MERGED

MicroServe

a build plan with the theory built in

Six milestones, each paired with exactly the theory it needs — read just before you need it, never before, never more than required.

MILESTONES	DURATION	HARDWARE	SERVING LAYER	COMPILED
6 + setup	6–10 weeks	Laptop GPU	Deferred	Aug 2026

Why interleaved, not linear

Reading all six inference-engineering phases before writing a line of code is the weaker strategy, for the same reason the generator detour was kept short before asyncio: theory read right before you need it sticks; theory read weeks ahead of any application mostly evaporates. So this plan doesn't separate "the roadmap" from "the project" — each milestone below names the exact reading it needs, immediately before it's needed, and just as importantly, names what to explicitly skip so the milestone doesn't balloon into a detour.

Two rules hold across every milestone. **Correctness before performance:** every milestone must still pass a fixed-seed, greedy-decode correctness test against plain HuggingFace `generate()` before its benchmark numbers count for anything. **Depth is capped deliberately:** each milestone has an explicit guardrail naming what's out of scope — not because those topics aren't interesting, but because going past the line turns a two-month systems project into a six-month kernel-authoring project.

MASTER MAP

	MILESTONE	BUILD	LEARN FIRST / SKIP
0	Setup	Repo, tiny model, correctness harness	*Nothing new*
1	Naive baseline	One-request-at-a-time engine	*Nothing new*
2	Continuous batching	Scheduler, iteration-level scheduling	Phase 2, Orca paper
3	Paged KV cache	Block allocator, block table, gather-attend	Phase 1, Part A1, PagedAttention paper
4	Quantization	4-bit loading, re-benchmark	Phase 3 quantization section
5	Benchmark suite	Poisson workload, 4-way comparison	Phase 6's TTFT/TPOT framing
6	Prefix sharing	Reference-counted blocks, copy-on-write	PagedAttention §4 (sharing)

> **DEFERRED ENTIRELY — NOT READ AT ANY POINT IN THIS PLAN**
>
> Phase 5 (distributed inference), Part A3 (vector indexes/Redis), and any CUDA/Triton kernel-authoring or Tensor Core instruction-level material. Each has a specific future trigger named in the appendix — none of them belong in this build.

MILESTONE ZERO

00

Setup and the correctness harness

Everything after this milestone is judged against the test you write here — get it right once, trust it for the rest of the project.

BUILD

Repo skeleton (`engine/` , `bench/` , `tests/`). A tiny model running through plain HF `generate()` — Qwen2.5-0.5B or Llama-3.2-1B, chosen specifically to keep your iteration loop fast while you're still debugging, not for realism. A correctness test that fixes the random seed, runs greedy decoding, and stores the exact output tokens as a golden reference.

LEARN FIRST

Nothing new. This milestone is pure engineering setup — the payoff of everything already covered in your OS/async roadmap.

GUARDRAIL

Don't install vLLM, Triton, or TensorRT-LLM yet. Don't pick the eventual 7B model yet — that decision belongs to Milestone 4, once quantization is in place and it actually fits your VRAM.

DEFINITION OF DONE

`pytest tests/test_correctness.py` passes, comparing your engine's output tokens to HF's reference output, identical seed, identical prompt.

MILESTONE ONE

01

Naive baseline

Deliberately bad, on purpose — this is the "before" number every later milestone is measured against.

BUILD

`engine.py` v0: a minimal `Sequence` object, one request handled at a time, wrapping HF's own generation loop and cache. No scheduler, no custom KV management yet.

LEARN FIRST

Nothing new — you already have everything this milestone needs.

GUARDRAIL

Resist building the scheduler or block allocator here even if it's tempting to get ahead — this milestone exists solely to produce a comparison baseline, and adding complexity now just makes that baseline murkier.

DEFINITION OF DONE

Correctness test passes. Throughput and latency recorded and saved — this number appears on every chart from Milestone 5 onward.

MILESTONE TWO

02

Continuous batching

The scheduling half of what makes vLLM fast — isolated here, deliberately, before memory management enters the picture in Milestone 3.

BUILD

A real `Sequence` with waiting/running/finished state, and a `Scheduler` implementing iteration-level scheduling — evicting finished sequences and admitting new ones every decode step, not every full batch. Run prefill (new sequences) and decode (running sequences) as two separate forward calls rather than fusing them — a deliberate simplification, noted as such in your README. Each sequence still reserves a simple contiguous KV buffer; fragmentation is expected and fine here.

LEARN FIRST

Phase 2 — prefill vs. decode. Prefill processes the whole prompt in one parallel pass (compute-bound); decode generates one token at a time, memory-bandwidth bound. You need this distinction precisely, because it's the reason prefill and decode get separate code paths in your scheduler.

Orca paper (via the Anyscale continuous batching writeup) — iteration-level scheduling, and why static batching collapses under variable output lengths.

`anyscale.com/blog/continuous-batching-llm-inference`

GUARDRAIL

Do not implement paged or block-based memory yet — accepting fragmentation here is correct, not lazy; it isolates the scheduling problem so Milestone 3 can isolate the memory problem. Do not attempt to fuse prefill and decode into one kernel call — that's a real vLLM optimization, and re-deriving it isn't necessary to prove you understand iteration-level scheduling.

DEFINITION OF DONE

Correctness test still passes. A benchmark demonstrating continuous batching beating static batching on throughput and GPU idle time under a workload with variable output lengths.

MILESTONE THREE · THE BIG ONE

03

Paged KV cache

Budget the most time here. This is the one line on the resume almost nobody else building a portfolio project has actually implemented.

BUILD

A `BlockAllocator` managing one pre-allocated KV tensor pool (shape `[num_blocks, 2, num_layers, num_heads, block_size, head_dim]`), with a free-block stack and reference counts. Each `Sequence` gets a `block_table` mapping logical position to physical block id. At attention time: **gather the sequence's scattered blocks into a contiguous scratch tensor, then call `scaled_dot_product_attention`** on it — no custom kernel required.

LEARN FIRST

Phase 1 — GPU memory hierarchy. Modal GPU Glossary, *Device Hardware* and *Device Software* sections only (SM, warp, thread/block/grid, the memory hierarchy) — not the host-side API pages. Then Horace He's *Making Deep Learning Go Brrrr* for the roofline model and the SRAM/HBM bandwidth gap.

Part A1 — virtual memory and paging. OSTEP Ch 13, 15, 18–19, 21–22. This is not an analogy exercise — PagedAttention's block table *is* an OS page table, applied to KV cache.

PagedAttention paper (Kwon et al., 2023) — read fully; it's short and every design choice in this milestone traces back to it.

`modal.com/gpu-glossary` · `horace.io/brrr_intro.html` · `pages.cs.wisc.edu/~remzi/OSTEP/` · `arxiv.org/abs/2309.06180`

GUARDRAIL — THE ONE MOST WORTH READING TWICE

Do **not** write a custom CUDA or Triton kernel that reads scattered blocks directly from memory — gather-then-attend with PyTorch's own `scaled_dot_product_attention` proves the same idea (dynamic, non-contiguous, on-demand allocation with a block table) without requiring kernel authorship, which is a separate, much larger project. Do **not** go into Tensor Core instruction-level behavior, PTX/SASS, warp-level programming, or occupancy tuning — none of that is required to explain or implement paging. Read only the two named Modal Glossary sections, not the site linearly end to end.

DESIGN DECISIONS ALREADY MADE

- One giant pre-allocated tensor pool, never per-block allocate/free on the GPU — allocator overhead would otherwise swamp your benchmark numbers.
- Block size is a tunable constant (start at 16 tokens/block, matching vLLM's default, and treat it as a parameter you can sweep later).

DEFINITION OF DONE

Correctness test passes on the paged code path. A memory-utilization benchmark showing higher achievable concurrent batch size at the same VRAM budget, contiguous allocation vs. paged.

MILESTONE FOUR

04

Quantization

The interesting result isn't "uses less memory" — it's how many more concurrent sequences your paged allocator now fits in the same VRAM.

BUILD

4-bit weight loading via `bitsandbytes` (NF4). Re-run every benchmark from Milestones 1–3 under quantization. This is also the milestone where you pick and load the real target model — a 7B-class model, 4-bit, once your Legion arrives.

LEARN FIRST

Phase 3 — quantization section only. What "weight-only post-training quantization" means, and the names GPTQ and AWQ, at the level of knowing what `bitsandbytes` is doing and why reducing bytes-per-parameter speeds up a memory-bandwidth-bound workload specifically.

GUARDRAIL

Do not implement your own quantization kernel. Do not study GPTQ's calibration procedure or AWQ's activation-aware weight-selection algorithm in depth — you are a *consumer* of these techniques in this project, not an implementer of them.

DEFINITION OF DONE

A chart showing memory footprint before/after quantization, and the resulting increase in maximum concurrent sequences at fixed VRAM — this is the number that ties Milestones 3 and 4 together.

MILESTONE FIVE

05

Benchmark suite

This is the artifact people actually look at. A clean chart earns more credibility than any paragraph describing the project.

BUILD

`bench/workload.py` — request arrivals drawn from a **Poisson process** (the standard choice in the Orca and vLLM papers themselves, not an arbitrary pick), with prompt/output lengths sampled from a small ShareGPT-derived set rather than uniform random — a specific, citable line for your README. `bench/benchmark.py` — drives `engine.step()` under this workload and records four numbers: system throughput, TTFT, TPOT, p50/p99 latency. Plot all four across your four engine versions (naive → batched → paged → quantized) at increasing request rate, one chart.

LEARN FIRST

Phase 6 — TTFT/TPOT framing only. Time-to-first-token reflects prefill; time-per-output-token reflects decode. That's the whole reading requirement — you don't need the rest of Phase 6 (the FastAPI/serving material) until serving becomes Act Two.

GUARDRAIL

Don't build a full statistical analysis pipeline or a dashboard. Four metrics, four engine versions, one chart, is enough to make the point and is what a reviewer will actually look at.

DEFINITION OF DONE

A README-ready comparison chart plus the raw numbers behind it, committed to the repo alongside the benchmark script that produced them.

MILESTONE SIX · STRETCH

06

Prefix / block sharing

Still core-engine work, not serving — the natural extension of Milestone 3's block allocator, and PagedAttention's other headline idea.

BUILD

Reference-counted blocks: when two sequences share an identical prompt prefix, they point at the same physical blocks instead of duplicating them. Copy-on-write when a shared block needs to diverge. A simple prefix-matching check on incoming requests to detect the sharing opportunity.

LEARN FIRST

Re-read the PagedAttention paper's memory-sharing / copy-on-write section specifically — you've already read the paper in Milestone 3, so this is a focused re-read, not new material.

GUARDRAIL

Do not build the RAG layer, a vector index, or any retrieval logic here — that's explicitly Act Two, later. This milestone is only about block reuse inside the engine you already have.

DEFINITION OF DONE

A measured reduction in prefill time on a workload with repeated or overlapping prompt prefixes, compared to Milestone 4's non-sharing baseline.

07

What's deferred, and its future trigger

Not cut permanently — each of these has a specific future moment where it becomes relevant again.

Phase 5 — distributed inference (tensor/pipeline parallelism). Trigger: a future project involving a model too large for one GPU. Irrelevant on a single laptop GPU regardless of which one you have.

Part A3 — vector indexes, Redis. Trigger: Act Two, when a RAG layer goes on top of this engine. Reading it now would be exactly the premature breadth this plan is designed to avoid.

Phase 4 — installing and running vLLM as a black box. Repurposed, not skipped: read vLLM's actual scheduler and block-manager source during Milestone 3 as a design reference for your own simplified version, rather than operating it as a tool.

CUDA/Triton kernel authoring, Tensor Core internals, PTX/SASS, occupancy tuning. Trigger: a future moment where profiling reveals a specific bottleneck that genuinely requires a custom kernel to fix — not before. This is a distinct specialization from what this project demonstrates.

FastAPI / HTTP serving layer, streaming, cancellation. Act Two, explicitly deferred per your own earlier call — becomes a thin wrapper once the engine here is done and benchmarked.

QUICK LINKS

OSTEP	`pages.cs.wisc.edu/~remzi/OSTEP/`
Modal GPU Glossary	`modal.com/gpu-glossary`
Making DL Go Brrrr — Horace He	`horace.io/brrr_intro.html`
PagedAttention / vLLM paper	`arxiv.org/abs/2309.06180`
vLLM source (reference during M3)	`github.com/vllm-project/vllm`
Continuous batching — Anyscale	`anyscale.com/blog/continuous-batching-llm-inference`

Every milestone above names its own finish line. If the timeline slips anywhere, let it slip on Milestone 6 — never on Milestone 3. That's the one nobody else has, and it's worth the extra week.